

计算机系统导论笔记

ICS Note

作者: Sam
邮箱: 2300012435@stu.pku.edu.cn
创建: 2025 年 9 月 29 日
更新: 2025 年 11 月 24 日

目录

1 虚拟内存	2
1.1 物理和虚拟寻址	2
1.2 虚拟内存作为缓存的工具	2
1.2.1 页表	2
1.2.2 页命中	3
1.2.3 缺页	4
1.2.4 分配页面	4
1.3 虚拟内存作为内存管理的工具	4
1.4 虚拟内存作为内存保护的工具	5
1.5 地址翻译	5
1.5.1 结合高速缓存和虚拟内存	6
1.5.2 利用 TLB 加速地址翻译	6
1.5.3 多级页表	7

1 虚拟内存

1.1 物理和虚拟寻址

在一个带虚拟内存的系统中，CPU 从一个有 $N = 2^n$ 个地址的地址空间中生成虚拟地址，这个地址空间称为虚拟地址空间 (virtual address space)。

一个系统还有一个物理地址空间 (physical address space)，对应于系统中物理内存的 M 个字节。

计算机系统的主存被组织成一个由 M 个连续的字节大小的单元组成的数组。每字节都有一个唯一的物理地址 (Physical Address, PA)。CPU 访问内存的最自然的方式就是使用物理地址。我们把这种方式称为物理寻址 (physical addressing)。

另一种方式为虚拟寻址 (virtual addressing)。CPU 通过生成一个虚拟地址 (Virtual Address, VA) 来访问主存，这个虚拟地址在被送到内存之前先转换成适当的物理地址。将一个虚拟地址转换为物理地址的任务叫做地址翻译 (address translation)。就像异常处理一样，地址翻译需要 CPU 硬件和操作系统之间的紧密合作。CPU 芯片上叫做内存管理单元 (Memory Management Unit, MMU) 的专用硬件，利用存放在主存中的查询表来动态翻译虚拟地址，该表的内容由操作系统管理。

1.2 虚拟内存作为缓存的工具

概念上而言，虚拟内存被组织为一个由存放在磁盘上的 N 个连续的字节大小的单元组成的数组。每字节都有一个唯一的虚拟地址，作为到数组的索引。磁盘上数组的内容被缓存在主存中。和存储器层次结构中其他缓存一样，磁盘（较低层）上的数据被分割成块，这些块作为磁盘和主存（较高层）之间的传输单元。VM 系统通过将虚拟内存分割为称为虚拟页 (Virtual Page, VP) 的大小固定的块来处理这个问题。每个虚拟页的大小为 $P = 2^p$ 字节。类似地，物理内存被分割为物理页 (Physical Page, PP)，大小也为 P 字节（物理页也被称为页帧 (page frame)）。

在任意时刻，虚拟页面的集合都分为三个不相交的子集：

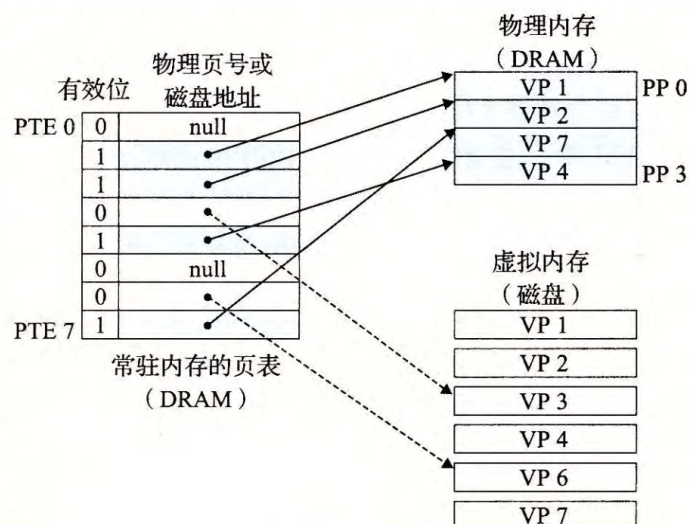
- 未分配的：VM 系统还未分配（或者创建）的页。未分配的块没有任何数据和它们相关联，因此也就不占用任何磁盘空间。
- 缓存的：当前已缓存在物理内存中的已分配页。
- 未缓存的：未缓存在物理内存中的已分配页。

1.2.1 页表

同任何缓存一样，虚拟内存系统必须有某种方法来判定一个虚拟页是否缓存在 DRAM 中的某个地方。如果是，系统还必须确定这个虚拟页存放在哪个物理页中。如果

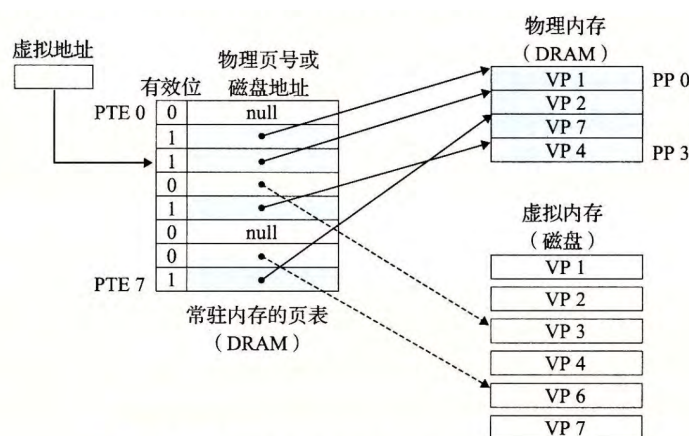
不命中，系统必须判断这个虚拟页存放在磁盘的哪个位置，在物理内存中选择一个牺牲页，并将虚拟页从磁盘复制到 DRAM 中，替换这个牺牲页。

这些功能是由软硬件联合提供的，包括操作系统软件、MMU（内存管理单元）中的地址翻译硬件和一个存放在物理内存中叫做页表 (page table) 的数据结构，页表将虚拟页映射到物理页。每次地址翻译硬件将一个虚拟地址转换为物理地址时，都会读取页表。操作系统负责维护页表的内容，以及在磁盘与 DRAM 之间来回传送页。



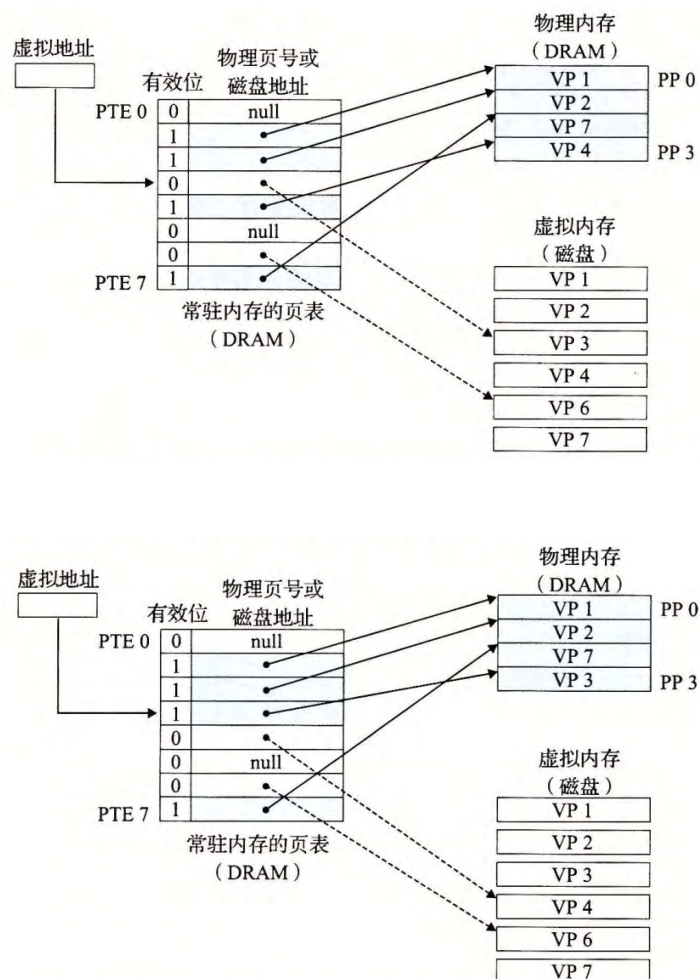
页表就是一个页表条目 (Page Table Entry, PTE) 的数组。虚拟地址空间中的每个页在页表中一个固定偏移处都有一个 PTE。每个 PTE 是由一个有效位 (valid bit) 和一个 n 位地址字段组成的。有效位表明了该虚拟页当前是否被缓存在 DRAM 中。如果设置了有效位，那么地址字段就表示 DRAM 中相应的物理页的起始位置，这个物理页中缓存了该虚拟页。如果没有设置有效位，那么一个空地址表示这个虚拟页还未被分配，这个地址就指向该虚拟页在磁盘上的起始位置。

1.2.2 页命中



1.2.3 缺页

在虚拟内存的习惯说法中，DRAM 缓存不命中称为缺页 (page fault)。缺页会触发一个缺页异常，缺页异常调用内核中的缺页异常处理程序。当异常处理程序返回时，它会重新启动导致缺页的指令，该指令会把导致缺页的虚拟地址重发送到地址翻译硬件。



当有不命中发生时，才换入页面的这种策略称为按需页面调度 (demand paging)。

1.2.4 分配页面

当操作系统分配一个新的虚拟内存页时，分配过程是在磁盘上创建空间并更新相应的 PTE，使它指向磁盘上这个新创建的页。

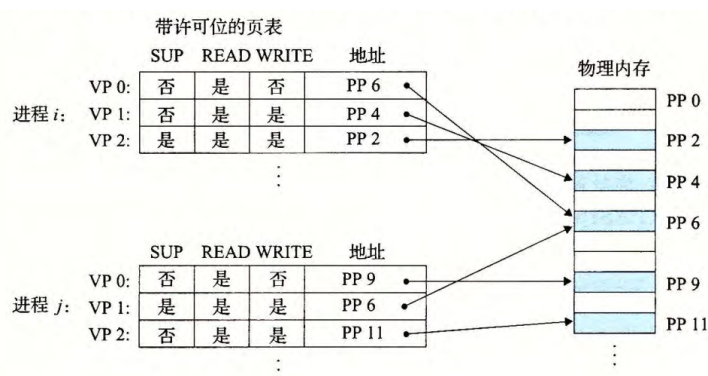
1.3 虚拟内存作为内存管理的工具

操作系统为每个进程提供了一个独立的页表，也就是一个独立的虚拟地址空间。多个虚拟页面可以映射到同一个共享物理页面上。

- 简化链接：独立的地址空间允许每个进程的内存映像使用相同的基本格式，而不管代码和数据实际存放在物理内存的何处。即不用考虑物理地址 PA，只用考虑虚拟地址 VA。
- 简化加载：要把目标文件中 .text 和 .data 节加载到一个新创建的进程中，Linux 加载器为代码和数据段分配虚拟页，把它们标记为无效的（即未被缓存的），将页表条目指向目标文件中适当的位置。
- 简化共享：不同进程中的虚拟页面可以映射到相同的物理页面。
- 简化内存分配：虚拟内存为向用户进程提供一个简单的分配额外内存的机制。当一个运行在用户进程中的程序要求额外的堆空间时（如调用 malloc 的结果），操作系统分配一个适当数字（例如 k）个连续的虚拟内存页面，并且将它们映射到物理内存中任意位置的 k 个任意的物理页面。由于页表工作的方式，操作系统没有必要分配 k 个连续的物理内存页面。页面可以随机地分散在物理内存中。

1.4 虚拟内存作为内存保护的工具体

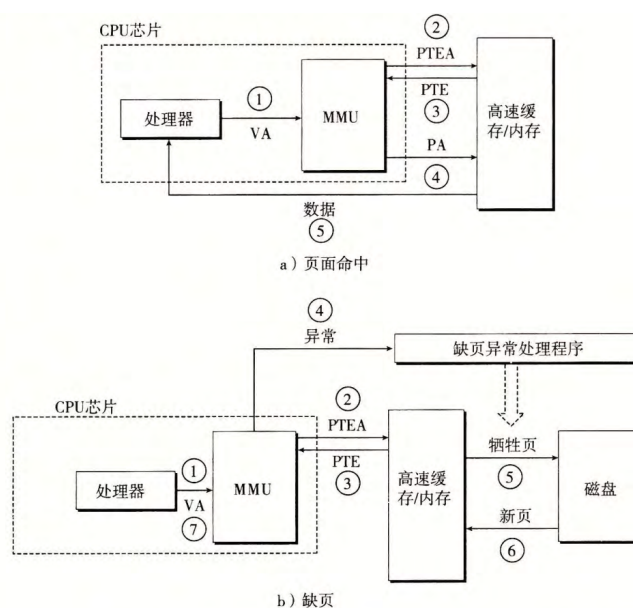
每次 CPU 生成一个地址时，地址翻译硬件都会读一个 PTE，通过在 PTE 上添加一些额外的许可位可以实现对一个虚拟页面内容访问的控制。



如果一条指令违反了这些许可条件，那么 CPU 就触发一个一般保护故障，将控制传递给一个内核中的异常处理程序。Linux shell 一般将这种异常报告为段错误 (segmentation fault)。

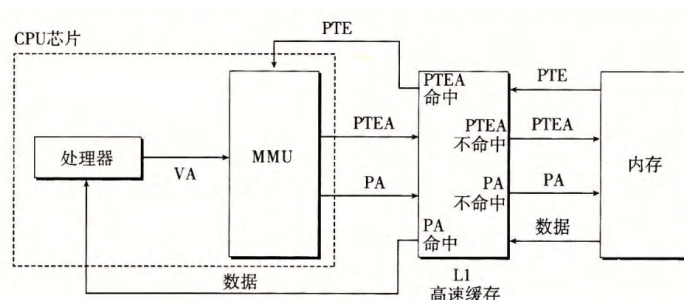
1.5 地址翻译

CPU 中的一个控制寄存器，页表基址寄存器 (Page Table Base Register, PTBR) 指向当前页表。n 位的虚拟地址包含两个部分：一个 p 位的虚拟页面偏移 (Virtual Page Offset, VPO) 和一个 (n - p) 位的虚拟页号 (Virtual Page Number, VPN)。MMU 利用 VPN 来选择适当的 PTE。然后将页表条目中物理页号 (Physical Page Number, PPN) 和虚拟地址中的 VPO 串联起来，就得到相应的物理地址。



1.5.1 结合高速缓存和虚拟内存

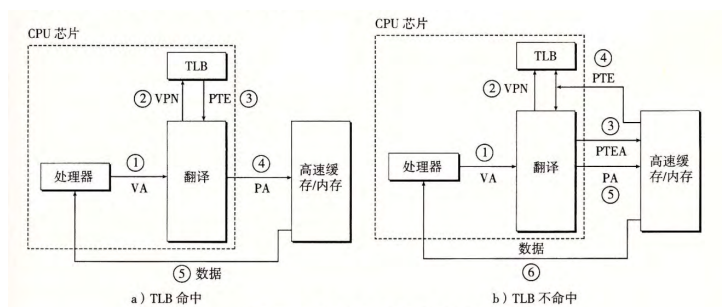
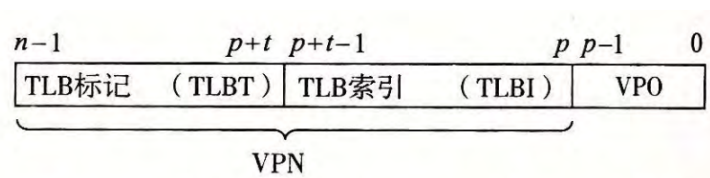
让地址翻译发生在高速缓存查找之前，便可以实现物理寻址的高速缓存和虚拟内存的结合。注意，页表条目可以缓存，就像其他的数据字一样。



1.5.2 利用 TLB 加速地址翻译

每次 CPU 产生一个虚拟地址，MMU 就必须查阅一个 PTE，以便将虚拟地址翻译为物理地址。在最糟糕的情况下，这会要求从内存多取一次数据，代价是几十到几百个周期。如果 PTE 碰巧缓存在 L1 中，那么开销就下降到 1 个或 2 个周期。然而，许多系统都试图消除即使是这样的开销，它们在 MMU 中包括了一个关于 PTE 的小的缓存，称为翻译后备缓冲器 (Translation Lookaside Buffer, TLB)。

TLB 是一个小的、虚拟寻址的缓存，其中每一行都保存着一个由单个 PTE 组成的块。TLB 通常有高度的相联度。用千组选择和行匹配的索引和标记字段是从虚拟地址中的虚拟页号中提取出来的。如果 TLB 有 $T = 2^t$ 个组，那么 TLB 索引 (TLBD) 是由 VPN 的 t 个最低位组成的，而 TLB 标记 (TLBT) 是由 VPN 中剩余的位组成的。



1.5.3 多级页表